

گزارش تمرین کامپیوتری اول هوش مصنوعی

امیرحسین صباحی

810101556

کدها:

game.py

کلاس PacmanGame وضعیت کامل بازی را نگه می‌دارد. موقعیت بازیکن، ارواح، میوه‌ها و دیوارها را ذخیره کرده و توابعی برای بررسی مرزها، هدف، و تولید حالت‌های بعدی دارد. در `get_next_states` همه حرکات ممکن بررسی می‌شود و فقط حالت‌های معتبر بدون برخورد با روح و دیوار بازگردانده می‌شود. این فایل مدل اصلی مسئله است که توسط تمام الگوریتم‌ها استفاده می‌شود.

bfs_solver.py

جست‌وجوی پهنا را با صف انجام می‌دهد. از حالت اولیه شروع کرده و همه حالت‌ها را به ترتیب عمق گسترش می‌دهد تا به هدف برسد. اولین مسیری که پیدا می‌کند کوتاه‌ترین است چون همه گام‌ها هزینه برابر دارند. حافظه زیاد مصرف می‌کند ولی همیشه مسیر بهینه می‌دهد.

dfs_solver.py

از پشته برای پیمایش عمقی استفاده می‌کند و تا رسیدن به هدف یا بن‌بست پیش می‌رود. حافظه کمی نیاز دارد اما ممکن است به شاخه‌های طولانی و بی‌فایده برود و جواب غیربهینه بدهد. با مجموعه `visited` از تکرار حالت‌ها جلوگیری می‌کند.

ids_solver.py

ترکیب DFS و BFS است. در هر مرحله تا عمق مشخصی جست‌وجو می‌کند و عمق را به تدریج زیاد می‌کند. این کار حافظه را پایین نگه می‌دارد ولی تضمین می‌کند اولین جواب کوتاه‌ترین مسیر باشد. نسخه پیاده‌شده چرخه‌ها را حذف می‌کند و از تکرار حالت‌ها جلوگیری می‌کند تا سرعت بهتر شود.

heuristics.py

تابع‌های تخمین هزینه (heuristic) را برای A^* تعریف می‌کند. h_zero همیشه صفر است (معادل دیکسترا) $h_max_to_target$ فاصله منتهن از بازیکن تا دورترین میوه موردنیاز را می‌دهد $h_mst_plus_min$ فاصله نزدیک‌ترین هدف به علاوه طول درخت پوشای کمینه بین تمام میوه‌ها را حساب می‌کند که تخمینی قوی‌تر و دقیق‌تر است. دو تابع $DEFAULT_HEURISTIC$ و $ALT_HEURISTIC$ برای انتخاب پیش‌فرض تعیین شده‌اند.

astar_solver.py

الگوریتم A^* را اجرا می‌کند و در واقع نسخه وزن‌دار را با وزن ۱ صدا می‌زند. از صف اولویت استفاده می‌کند و $f(n)=g(n)+h(n)$ را مبنا قرار می‌دهد تا همیشه مسیر بهینه پیدا کند.

weighted_astar_solver.py

همان A^* است ولی در تابع f از ضریب α برای کنترل اهمیت heuristic استفاده می‌شود، $f=g+\alpha \times h$. اگر $\alpha=1$ باشد همان A^* معمولی است؛ اگر بزرگ‌تر شود الگوریتم سریع‌تر ولی احتمالاً غیر بهینه می‌شود. این نسخه بین سرعت و دقت تعادل برقرار می‌کند و از heuristic های فایل `heuristics.py` استفاده می‌کند.

پاسخ سوالات:

1. state همان نقشه‌ی کامل بازی در هر لحظه است ؛ یعنی آرایه‌ای دو بعدی از کاراکترها که

نشان می‌دهد در هر خانه چه چیزی وجود دارد.

ایراد این تعریف این است که بیشتر بخش‌های نقشه (مثل دیوارها) در طول بازی تغییر نمی‌کنند، ولی در این تعریف در هر حالت دوباره ذخیره و مقایسه می‌شوند. در نتیجه حجم حالت‌ها بسیار زیاد می‌شود و مقایسه‌ی دو state هم زمان‌بر است.

برای رفع این مشکل باید فقط قسمت‌های متغیر بازی را در تعریف state نگه داشت؛ یعنی موقعیت بازیکن، موقعیت و جهت حرکت روح‌ها، و وضعیت میوه‌ها (موقعیت و اینکه خورده شدن یا نه). به این ترتیب دو حالتی که از نظر منطق بازی یکسان‌اند، دقیقاً یک state محسوب می‌شوند، و الگوریتم‌های جست‌وجو می‌توانند سریع‌تر و با حافظه‌ی کمتر عمل کنند.

2. با تعریفی که از state داشتیم (مکان بازیکن، روح‌ها و میوه‌ها)، action یعنی کاری که

بازیکن در هر لحظه می‌تواند انجام دهد.

در این بازی، action ها همان حرکات‌های بالا، پایین، چپ و راست هستند. یعنی بازیکن در هر لحظه می‌تواند یکی از چهار جهت U، D، L، R را انتخاب کند.

وقتی یک action انجام می‌شود، بازیکن یک خانه در آن جهت حرکت می‌کند، روح‌ها هم طبق قانون خودشون یک خانه جابه‌جا می‌شوند. اگر بازیکن به دیوار بخورد یا با روح برخورد کند، آن حرکت معتبر نیست. اگر بعد از حرکت روی یک میوه‌ی مجاز بایستد، آن میوه خورده می‌شود. در همه‌ی حالت‌ها هزینه هر حرکت برابر ۱ در نظر گرفته می‌شود.

3

برای مدل کردن این مسئله باید اجزای اصلی یک مسئله‌ی جست‌وجو را مشخص کنیم، یعنی: initial state, goal state, action, transition model, و cost function.

1. initial state همان وضعیت شروع بازی است؛ جایی که مکان اولیه‌ی بازیکن (P)، موقعیت و جهت روح‌ها و میوه‌ها مشخص است. دیوارها ثابت‌اند و جزئی از محیط محسوب می‌شوند، نه بخشی از حالت متغیر.
2. goal state حالتی است که در آن همه‌ی میوه‌ها خورده شده‌اند. طبق قاعده‌ی بازی، بازیکن باید اول تمام میوه‌های A را بخورد و بعد سراغ B ها برود؛ بنابراین وقتی هیچ میوه‌ی A و B باقی نمانده باشد، به هدف رسیده‌ایم.
3. Action حرکت‌های ممکن بازیکن روی شبکه‌اند: بالا (U)، پایین (D)، چپ (L)، راست (R). هر حرکت فقط زمانی مجاز است که به دیوار نخورد و باعث برخورد با روح نشود.
4. transition model با انجام یک action، بازیکن و روح‌ها هر دو یک گام حرکت می‌کنند. اگر بازیکن روی یک میوه‌ی مجاز (A یا B بسته به مرحله) برود، آن میوه از بین می‌رود. اگر با روح برخورد کند، آن مسیر شکست‌خورده است و آن state معتبر نیست.
5. cost function برای سادگی، هزینه‌ی هر حرکت را برابر ۱ در نظر می‌گیریم. هدف، یافتن مسیری با کمترین مجموع هزینه (یعنی کوتاه‌ترین مسیر ایمن تا خوردن همه‌ی میوه‌ها) است.

4.

در این بازی فضای حالت خیلی بزرگ است، چون در هر لحظه بازیکن می‌تواند در چهار جهت حرکت کند، روح‌ها هم جابه‌جا می‌شوند و وضعیت میوه‌ها هم تغییر می‌کند. اگر بخواهیم در هر گام همه حالت‌های ممکن را گسترش بدهیم. تعداد حالت‌ها به صورت نمایی زیاد می‌شود. این باعث می‌شود حافظه و زمان اجرا خیلی زیاد شود و الگوریتم عملاً نتواند جواب بدهد. برای کم کردن این فضا و بهبود مسئله چند کار می‌شود کرد:

1. هرس کردن حرکات بی‌فایده

مثلاً اگر حرکت باعث می‌شود بازیکن دوباره به موقعیت قبلی برگردد (رفت و برگشت بی‌دلیل)، آن را تولید نکنیم. یا اگر حرکتی به دیوار یا به داخل مسیر بن‌بست منتهی می‌شود، همان ابتدا حذف شود.

2. استفاده از حافظه‌ی حالت‌های بازدیدشده (visited)

در الگوریتم‌های مثل BFS و A^* ، وقتی یک حالت را بررسی کردیم، دوباره همان را گسترش نمی‌دهیم. این کار جلوی چرخه‌ها را می‌گیرد.

3. استفاده از heuristic خوب در A^*

heuristic تخمین فاصله تا هدف را می‌دهد و باعث می‌شود فقط مسیرهایی که احتمالاً بهترند زودتر گسترش پیدا کنند.

مثلاً استفاده از فاصله‌ی تا نزدیک‌ترین میوه یا جمع مینیمم درخت پوشا روی میوه‌ها باعث می‌شود از مسیرهای بی‌هدف دوری کنیم.

4. توقف زودهنگام

اگر بازی به وضعیتی رسید که ادامه‌ی آن دیگر هدف را بهتر نمی‌کند (مثلاً همه‌ی میوه‌ها خورده شده)، همان‌جا قطع کنیم و بقیه‌ی شاخه‌ها را رها کنیم.

در مورد expand هم لازم نیست همه‌ی حالت‌های ممکن را در هر قدم گسترش بدهیم. اگر این کار را بکنیم فضای جست‌وجو خیلی بزرگ می‌شود و حل مسئله عملاً غیرممکن می‌شود. با هرس کردن حرکات غیرممکن یا تکراری، نگهداری visited state ها، و استفاده از heuristic مناسب می‌توانیم سرعت و کارایی الگوریتم را خیلی بالا ببریم.

5

الگوریتم BFS

در BFS ابتدا حالت شروع بررسی می‌شود. بعد همه‌ی حالت‌های در فاصله‌ی 1 از آن سپس فاصله‌ی 2 و به همین ترتیب یعنی اول تمام مسیرهای کوتاه‌تر بررسی می‌شوند. مزیت این روش این است که همیشه کوتاه‌ترین مسیر را پیدا می‌کند وقتی هزینه‌ی حرکت‌ها برابر باشد. عیبش این است که حافظه‌ی زیادی مصرف می‌کند چون باید صفی از تمام حالت‌ها در هر عمق نگه دارد و زمان اجرا هم زیاد است چون همه‌ی مسیرها تا رسیدن به هدف بررسی می‌شوند.

الگوریتم DFS

در DFS تا جایی که می‌شود در یک مسیر جلو می‌رویم و وقتی به بن‌بست رسیدیم برمی‌گردیم و مسیر بعدی را امتحان می‌کنیم. مزیتش این است که حافظه‌ی کمی لازم دارد چون فقط مسیر جاری را نگه می‌دارد. عیبش این است که ممکن است در شاخه‌های بی‌فایده گیر کند و حتی به جواب نرسد و جواب پیدا شده هم همیشه بهینه نیست.

الگوریتم IDS

الگوریتم IDS ترکیبی از BFS و DFS است. اول DFS با عمق 1 اجرا می‌کند بعد با عمق 2 بعد 3 و همینطور تا رسیدن به جواب. مزیت آن این است که مثل BFS جواب بهینه می‌دهد چون از عمق کم شروع می‌کند و مثل DFS حافظه‌ی کمی می‌خواهد. عیبش این است که بعضی گره‌ها چند بار تکراری بررسی می‌شوند پس زمانش کمی بیشتر از BFS است.

الگوریتم A*

در A* از تابع f برابر g بعلاوه‌ی h استفاده می‌شود. g هزینه‌ی طی‌شده تا اینجا و h تخمین هزینه‌ی باقی‌مانده تا هدف است که از heuristic می‌آید. الگوریتم حالتی را گسترش می‌دهد که f کمتر است.

مزیتش این است که اگر heuristic درست انتخاب شود جواب بهینه می‌دهد و از BFS سریع‌تر است. عیبش نیاز به حافظه‌ی زیاد است چون باید همه‌ی حالت‌های باز را نگه دارد.

الگوریتم *WeightedA

این روش مثل A^* است ولی در تابع f روی h یک ضریب بزرگ‌تر از یک گذاشته می‌شود. به این ترتیب به تخمین آینده بیشتر اعتماد می‌کند. مزیتش سرعت بالاتر است چون حالت‌های کمتری گسترش می‌دهد. عیبش این است که ممکن است جواب دقیق بهینه نباشد چون تخمین بیش از حد تاثیر دارد.

a. مشکل اصلی الگوریتم DFS این است که ممکن است در مسیرهای بی‌فایده خیلی عمیق پایین برود و هیچ‌وقت به جواب نرسد به‌خصوص اگر فضای حالت بزرگ یا بی‌نهایت باشد همچنین جواب پیدا شده لزوماً کوتاه‌ترین مسیر نیست چون DFS فقط تا اولین جواب ادامه می‌دهد و مسیرهای دیگر را بررسی نمی‌کند.

با این حال از DFS هنوز استفاده می‌شود چون حافظه‌ی بسیار کمی لازم دارد و پیاده‌سازی آن ساده است در مسائلی که عمق جواب زیاد نیست یا فقط پیدا کردن یک جواب کافی است DFS انتخاب مناسبی است.

b. الگوریتم IDS یا جست‌وجوی عمق‌افزایشی ترکیبی از مزیت‌های BFS و DFS است. این الگوریتم چند بار DFS را اجرا می‌کند اما هر بار عمق مجاز جست‌وجو را یکی افزایش می‌دهد مثلاً اول تا عمق یک می‌رود بعد تا عمق دو بعد سه و همین‌طور تا وقتی به جواب برسد. پس مثل BFS از عمق کم شروع می‌کند پس اولین جواب پیدا شده کوتاه‌ترین مسیر است و در نتیجه جواب بهینه خواهد بود و مثل DFS در هر مرحله فقط مسیر فعلی را نگه می‌دارد و حافظه‌ی کمی نیاز دارد. IDS در واقع برای حل دو مشکل اصلی طراحی شده است یکی نیاز زیاد به حافظه در BFS و دیگری نرسیدن به جواب یا بهینه نبودن جواب در DFS.

6.

برای مثال در نقشه ششم

BFS بیشترین تعداد حالت دیده شده و بیشترین حافظه را دارد ولی جواب کاملاً بهینه می دهد. 47
DFS کمترین حافظه را دارد اما ممکن است در مسیر اشتباه حرکت کند و جوابش طولانی یا ناقص باشد. 92

IDS با حافظه کم جواب بهینه می دهد ولی زمانش کمی بیشتر از BFS است چون بعضی حالت ها چند بار بررسی می شوند. 71

A^* معمولاً سریع تر از BFS است و تعداد حالت های گسترش یافته اش کمتر است چون از heuristic برای هدایت استفاده می کند و همچنان جواب بهینه می دهد. 41
 $Weighted A^*$ از همه سریع تر است و تعداد حالت های دیده شده اش کم تر است اما ممکن است طول مسیر کمی بیشتر از حداقل باشد.
بنابراین الگوریتم هایی که جواب بهینه تولید می کنند IDS BFS و A^* هستند چون یا همه حالت ها را تا عمق جواب بررسی می کنند یا heuristic آنها تضمین می کند که مقدار تخمین کمتر از مقدار واقعی است.

7.

اگر مجموع فاصله ی منهتن از Pacman تا همه ی خوراکی ها را به عنوان heuristic بگیریم، مقدارش معمولاً بیشتر از هزینه ی واقعی می شود چون فرض می کند Pacman می تواند هم زمان به چند میوه برود در حالی که باید یکی یکی آنها را بخورد. به همین دلیل این heuristic بیش برآورد است و ممکن است جواب بهینه ندهد. برای اصلاح می توان به جای آن از فاصله ی Pacman تا نزدیک ترین میوه یا از مجموع فاصله ی درخت پوشای کمینه بین میوه ها استفاده کرد تا تخمین واقعی تر و درست تر شود.

8.

اگر مجموع فاصله‌ی منتهن Pacman تا همه‌ی میوه‌ها را به عنوان heuristic بگیریم، چون فرض می‌کند می‌تواند هم‌زمان به چند میوه برود مقدارش معمولاً بیشتر از هزینه‌ی واقعی می‌شود و دیگر admissible نیست برای اصلاح می‌توان از فاصله تا نزدیک‌ترین میوه استفاده کرد که همیشه از هزینه‌ی واقعی کمتر است و بنابراین admissible و consistent است.

heuristic دیگری که بهتر عمل می‌کند این است که علاوه بر فاصله‌ی Pacman تا نزدیک‌ترین میوه، فاصله‌ی بین میوه‌ها را هم در نظر بگیریم مثلاً با استفاده از مجموع فاصله‌ی درخت پوشای کمینه بین آن‌ها این روش تخمینی دقیق‌تر می‌دهد و همچنان admissible و consistent باقی می‌ماند.

در مقایسه با این‌ها اگر مقدار heuristic را همیشه صفر بگیریم الگوریتم A^* دقیقاً مثل الگوریتم Dijkstra رفتار می‌کند که هم admissible و هم consistent است و همیشه جواب بهینه را پیدا می‌کند اما کندتر است.

9.

h_{zero} کم سرعت‌تر و با Expanded زیادی است ولی همیشه جواب بهینه می‌دهد.
 $h_{max_to_target}$ از نظر Expanded کمتر از BFS است و جواب بهینه می‌دهد چون دیوارها را به‌عنوان حد پایین نادیده می‌گیرد و سازگار.
 $h_{mst_plus_min}$ بهترین تعادل را دارد؛ یعنی تعداد حالت‌های گسترش یافته کمتر است، زمان اجرای جست‌وجو بهتر (کوتاه‌تر) است، و در عین حال طول مسیر به‌دست‌آمده همچنان بهینه باقی می‌ماند.

10.

اگر heuristic فقط admissible باشد، الگوریتم A^* همچنان به جواب بهینه می‌رسد، ولی ممکن است بعضی حالت‌ها چند بار گسترش داده شوند، چون مقدار f در مسیرهای مختلف ممکن است کم و زیاد شود.

اما اگر heuristic علاوه بر admissible، consistent هم باشد، هر حالت فقط یک بار گسترش داده می‌شود و الگوریتم هم سریع‌تر و هم ساده‌تر عمل می‌کند.

11.

$h_mst_plus_min$ بیشترین تأثیر را در افزایش سرعت جست‌وجو دارد. چون این heuristic علاوه بر فاصله‌ی Pacman تا نزدیک‌ترین میوه، ارتباط بین همه‌ی میوه‌های باقی‌مانده را هم در نظر می‌گیرد و با استفاده از درخت پوشای کمینه (MST) تخمینی واقع‌بینانه از هزینه‌ی واقعی می‌دهد. به همین دلیل مسیرهای بی‌فایده زودتر حذف می‌شوند و الگوریتم فقط مسیرهایی را گسترش می‌دهد که احتمال رسیدن به هدف در آن‌ها بیشتر است.

Time Taken Per Seconds

Algorithm	BFS	DFS	IDS	A*	A*W
Map1	0	0	0.1	0	0
Map2	0	0	0	0	0
Map3	0.2	0.14	2.77	0.13	0.12
Map4	22.8	5.77		15.8	14.7
Map5	2.27	0.61	240	0	0
Map6	3.51	0.41	106	2.37	2.28
Map7	1.27	0.55		1.35	1.36
Map8	51.6	3.16		11.6	10.1
Map9	61.7	2.44		52.9	51.3
Map10	68.2	8.77		27.5	26.2

Frames

Algorithm	BFS	DFS	IDS	A*	A*W
Map1	12	12	12	12	12
Map2	2	2	2	2	2
Map3	34	60	58	34	34
Map4	236	510		236	238
Map5	97	158	97	97	97
Map6	47	92	71	47	47
Map7	209	237		209	209
Map8	101	214		101	101
Map9	120	481		120	120
Map10	93	419		93	93

✓ TERMINAL

PS F:\Term 7\AI\codes> & C:/Users/NoteBook/AppData/Local/Microsoft/WindowsApps/python3.12.exe "f:/Term 7/AI/codes/tester.py"
Results on map1:

Algorithm	Time	Numof Moves	Result
BFS	0.07	12	Success
DFS	0.02	12	Success
IDS	0.12	12	Success
A*	0.02	12	Success
Weighted A*	0.02	12	Success

Results on map2:

Algorithm	Time	Numof Moves	Result
BFS	0	2	Success
DFS	0	2	Success
IDS	0	2	Success
A*	0	2	Success
Weighted A*	0	2	Success

Results on map3:

Algorithm	Time	Numof Moves	Result
BFS	0.2	34	Success
DFS	0.14	60	Success
IDS	2.93	58	Success
A*	0.14	34	Success
Weighted A*	0.11	34	Success

Results on map4:

Algorithm	Time	Numof Moves	Result
BFS	22.57	236	Success
DFS	5.74	510	Success
IDS	17945	0	Timeout
A*	16.57	236	Success
Weighted A*	14.87	240	Success